

31

✎ 추가 일시	2026년 8월 20일 오후 8:10
🏠 강의	애플리케이션 보안

OWASP Juice Shop 취약점 분석 및 실습

1. 실습 개요

대상

- OWASP Juice Shop

목적

OWASP Juice Shop을 대상으로 다양한 웹 애플리케이션 취약점을 직접 분석하고, Burp Suite 및 Kali Linux 등의 도구를 활용하여 취약점이 실제 서비스에 어떤 영향을 미칠 수 있는지 확인한다.

주요 실습 항목

- 인증 정보 및 쿠키 노출
- SQL Injection
- 관리자 기능 접근
- 무차별 대입 공격(Brute Force)
- XSS
- 장바구니 수량 변조
- 다른 사용자의 장바구니 접근
- 파일 다운로드 취약점
- Easter Egg 탐색 및 인코딩 분석

2. 인증 정보 및 사용자 ID 노출

리뷰 작성 기능을 확인한 결과, 리뷰 작성 과정에서 사용자의 ID가 노출되는 것을 확인하였다.

또한 ID와 비밀번호가 쿠키 값에 평문 형태로 저장되는 것을 확인하였다.

확인된 문제

- 사용자 식별 정보가 외부에 노출될 가능성이 존재
- 인증 관련 정보가 쿠키에 평문으로 저장될 경우 탈취 시 계정 접근에 악용될 가능성이 존재
- 노출된 ID를 기반으로 비밀번호 공격을 시도할 수 있음

리뷰 작성 과정에서 두 개의 사용자 ID를 확보한 뒤 관리자 계정에 대한 로그인을 시도하였다.

보안상 의미

사용자 ID가 쉽게 노출되고 비밀번호 공격에 대한 방어가 부족하다면 공격자가 계정 탈취를 시도하기 쉬워진다.

대응 방안

- 민감한 인증 정보를 쿠키에 평문으로 저장하지 않는다.
- 인증 정보는 안전한 세션 관리 방식을 사용한다.
- 쿠키에 `HttpOnly`, `Secure`, `SameSite` 등의 보안 속성을 적용한다.
- 로그인 시도 횟수 제한 및 계정 잠금 정책을 적용한다.

3. SQL Injection을 이용한 관리자 로그인

관리자 로그인 페이지에서 로그인 ID 입력값에 SQL Injection을 시도하였다.

비밀번호에는 임의의 값을 입력한 상태에서 SQL 구문이 정상적으로 처리되는지 확인하였다.

그 결과 SQL Injection을 통해 **admin 계정으로 로그인하는 데 성공하였다.**

취약점 원인

사용자가 입력한 로그인 값이 SQL 쿼리에 안전하게 처리되지 않고 직접 반영될 경우 공격자가 SQL 쿼리의 논리를 변경할 수 있다.

영향

- 인증 우회
- 관리자 계정 접근
- 관리자 기능 악용
- 데이터 조회 및 변경 가능성

대응 방안

- Prepared Statement 사용
- Parameterized Query 사용
- 사용자 입력값 검증
- DB 계정에 최소 권한 적용
- 오류 메시지를 외부에 상세하게 노출하지 않도록 설정

4. 관리자 페이지 접근

관리자 계정으로 로그인한 이후 페이지 소스 코드를 분석하여 URL 정보를 확인하였다.

소스 코드에 포함된 URL 중 관리자 기능과 관련된 경로에 접근한 결과, Administration 세션에 접근할 수 있었다.

해당 관리자 페이지에서는 여러 사용자의 계정 정보를 확인할 수 있었으며, 사용자 리뷰를 삭제하는 등의 관리자 기능도 사용할 수 있었다.

보안상 영향

단순한 로그인 우회를 넘어 관리자 기능까지 접근할 수 있기 때문에 인증 및 접근제어가 제대로 구현되지 않은 경우 서비스 전체에 큰 영향을 줄 수 있다.

대응 방안

- 관리자 페이지에 대한 별도의 접근제어 적용
- 서버 측에서 사용자 권한을 매 요청마다 검증
- URL을 알고 있다는 이유만으로 관리자 기능에 접근할 수 없도록 구성
- 최소 권한 원칙 적용

5. 무차별 대입 공격(Brute Force)

관리자 계정의 비밀번호가 충분히 강력한지 확인하기 위해 Burp Suite의 Intruder 기능을 활용하여 무차별 대입 공격을 수행하였다.

Kali Linux에 비밀번호 사전을 설치한 후 다음 사전 파일을 사용하였다.

```
/usr/share/seclists/password/common-credentials/best1050.txt
```

Burp Suite의 Payload Configuration에 비밀번호 사전을 불러온 후 로그인 요청에 적용하였다.

응답 결과의 상태 코드 및 응답 차이를 비교하여 정상적으로 인증되는 요청을 식별하였다. 그 결과 다음 비밀번호를 확인할 수 있었다.

```
admin123
```

문제점

`admin123` 과 같이 추측하기 쉬운 비밀번호를 사용하고 있으며, 반복적인 로그인 시도를 제한하는 방어 기능이 부족한 것으로 판단된다.

대응 방안

- 강력한 비밀번호 정책 적용
- 로그인 실패 횟수 제한
- 계정 잠금 또는 일정 시간 로그인 제한
- CAPTCHA 적용
- MFA 적용
- 비밀번호 사전 공격 및 유출 비밀번호 차단

6. XSS 취약점

다음으로 사용자 입력값에 대한 XSS 취약점을 확인하였다.

기본적인 `<script>` 형태의 입력을 테스트하였으나 특정 문자가 필터링되는 것을 확인하였다.

이후 다른 HTML 요소를 이용하여 XSS가 실행되는지 확인하였으며, 결과적으로 XSS 실행에 성공하였다.

XSS 실행 이후 `document.cookie` 를 이용하여 현재 브라우저의 쿠키 정보가 출력되는 것도 확인하였다.

취약점 원인

사용자 입력값이 적절하게 검증 및 인코딩되지 않은 상태로 브라우저에서 HTML/JavaScript로 해석될 경우 공격자가 삽입한 스크립트가 실행될 수 있다.

영향

공격자의 스크립트가 다른 사용자의 브라우저에서 실행될 경우 다음과 같은 피해가 발생할 수 있다.

- 세션 정보 노출
- 사용자 정보 탈취
- 관리자 페이지 조작
- 악성 페이지로의 이동
- 사용자 대신 특정 기능 실행

대응 방안

- 출력 시 HTML Entity Encoding 적용
- 사용자 입력값 검증
- 불필요한 HTML 태그 제거
- Content Security Policy(CSP) 적용
- 쿠키에 `HttpOnly` 속성 적용

7. 장바구니 수량 변조

장바구니 기능을 확인하던 중 상품의 수량 값이 클라이언트 요청에 포함되는 것을 확인하였다.

애플 주스를 장바구니에 추가한 후 Burp Suite를 이용하여 요청을 확인하였다.

기존 수량이 1로 설정되어 있었으며 해당 값을 다른 값으로 변경하여 요청을 전송한 결과 장바구니의 수량이 변경되는 것을 확인하였다.

취약점 원인

서버에서 상품 수량 및 가격과 같은 중요한 값을 신뢰하고 검증하지 않는 경우 클라이언트 요청을 변조하여 비정상적인 주문을 생성할 수 있다.

대응 방안

- 상품 가격 및 수량을 서버에서 검증
- 클라이언트가 전달한 값을 그대로 신뢰하지 않는다.
- 주문 처리 시 DB의 실제 상품 정보를 기준으로 재계산
- 비정상적인 수량에 대한 서버 측 검증 적용

8. 주문 기능 확인

변조된 장바구니 정보를 기반으로 Checkout 과정을 진행하였다.

배송 주소 및 배송 옵션을 지정한 뒤 주문 과정까지 정상적으로 진행되는 것을 확인하였다.

이를 통해 장바구니 단계에서 변조된 값이 이후 주문 과정에도 영향을 줄 가능성이 있음을 확인하였다.

9. 다른 사용자의 장바구니 접근

Burp Suite에서 관리자 계정과 관련된 요청 값을 확인하던 중 사용자 식별 값이 요청에 포함되는 것을 확인하였다.

해당 값을 다른 값으로 변경하여 요청을 전송한 결과 다른 사용자의 장바구니 정보를 확인할 수 있었다.

취약점 유형

이는 사용자의 객체 식별값만 변경하여 다른 사용자의 데이터에 접근할 수 있는 **접근제어 취약점(IDOR/BOLA 계열)**으로 볼 수 있다.

영향

- 다른 사용자의 장바구니 조회
- 개인정보 노출 가능성
- 다른 사용자의 데이터 변경 가능성

대응 방안

서버에서 요청한 사용자와 요청 대상 객체의 소유 관계를 반드시 검증해야 한다.

```
요청 사용자
↓
인증 확인
↓
객체 ID 확인
↓
해당 객체의 소유자와 요청 사용자 비교
↓
일치하는 경우에만 접근 허용
```

단순히 URL이나 요청 파라미터의 ID가 유효한지만 확인해서는 안 된다.

10. 파일 다운로드 취약점

FTP 관련 페이지에서 `package.json` 파일에 접근을 시도하였다.

일반적인 접근 방식에서는 오류가 발생하여 파일 내용을 확인할 수 없었지만, URL에 인코딩된 값을 추가하여 요청한 결과 파일을 다운로드하고 내용을 확인할 수 있었다.

보안상 문제

웹 애플리케이션이 파일 경로 및 파일 접근 요청을 안전하게 처리하지 않는 경우 원래 의도하지 않았던 파일에 접근할 수 있다.

대응 방안

- 파일 접근 권한 검증
- 허용된 파일 목록(Allowlist) 적용
- 사용자 입력을 파일 경로에 직접 사용하지 않도록 구성
- URL 인코딩 및 디코딩 과정에서 우회가 발생하지 않도록 검증
- 서버 내부 중요 파일의 웹 접근 차단

11. Easter Egg 분석

FTP 페이지에서 Easter Egg와 관련된 페이지를 확인하였다.

페이지의 특정 내용이 정상적으로 표시되지 않았으나 파일 접근 과정에서 인코딩된 파일을 확인할 수 있었다.

해당 내용을 Base64 방식으로 디코딩한 결과 다음 문자열을 얻었다.

```
/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt
```

이 문자열을 다시 ROT13 방식으로 디코딩한 결과 다음 문장이 확인되었다.

```
/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg
```

해당 URL에 접근하여 최종 Easter Egg 페이지를 확인하였다.

학습 내용

이번 과정에서는 단순히 웹 취약점만 확인한 것이 아니라 다음과 같은 데이터 처리 방식도 확인하였다.

- URL Encoding
- Base64
- ROT13
- 파일 다운로드 및 접근
- 소스 코드 및 URL 분석

12. 취약점 정리

구분	취약점	주요 영향
인증	인증 정보 및 ID 노출	계정 공격 가능성
인증	SQL Injection	로그인 우회 및 관리자 접근
접근제어	관리자 기능 접근	관리자 기능 악용
인증	Brute Force	비밀번호 탈취
클라이언트	XSS	세션/정보 노출 가능
비즈니스 로직	장바구니 수량 변조	주문 정보 조작
접근제어	다른 사용자 장바구니 접근	사용자 데이터 노출
파일 처리	파일 다운로드 우회	내부 파일 노출
정보수집	소스/URL 분석	숨겨진 기능 및 정보 노출

13. 종합 분석

이번 OWASP Juice Shop 실습을 통해 웹 애플리케이션의 취약점은 하나의 기능에서만 발생하는 것이 아니라 **인증, 접근제어, 입력값 검증, 세션 관리, 파일 처리, 비즈니스 로직 등 여러 영역에서 발생할 수 있다는 점**을 확인하였다.

특히 SQL Injection을 통한 관리자 로그인, Brute Force를 통한 비밀번호 확인, XSS를 통한 쿠키 접근, 다른 사용자의 장바구니 접근 등의 실습을 통해 단순히 취약점을 발견하는 것보다 **취약점이 실제 서비스에 어떤 영향을 미치는지 분석하는 과정이 중요하다는 것을 확인**하였다.

또한 Burp Suite를 활용하여 브라우저와 서버 사이의 HTTP 요청을 직접 확인하고 파라미터를 변경해 보으로써, 웹 애플리케이션의 서버 측 검증이 얼마나 중요한지 확인할 수 있었다.

14. 보안 개선 방향

전체적으로 다음과 같은 보안 대책을 적용할 필요가 있다.

1. 입력값 검증

- 사용자 입력값에 대한 검증 및 필터링

2. 출력 인코딩

- XSS 방지를 위한 상황별 출력 인코딩 적용

3. 안전한 SQL 처리

- Prepared Statement 및 Parameterized Query 사용

4. 강력한 인증

- 강력한 비밀번호 정책
- MFA
- 로그인 시도 제한

5. 세션 보안

- HttpOnly
- Secure
- SameSite
- 적절한 세션 만료 정책

6. 접근제어 강화

- 모든 요청에 대해 서버 측 권한 검증
- 객체 소유권 검증
- 관리자 기능 별도 권한 검증

7. 서버 측 비즈니스 로직 검증

- 상품 가격 및 수량 등 중요 값을 클라이언트 요청에 의존하지 않도록 구성

8. 파일 접근 제어

- 허용된 파일만 접근할 수 있도록 Allowlist 적용
- 내부 파일의 외부 노출 차단

15. 결론

OWASP Juice Shop을 대상으로 다양한 웹 애플리케이션 취약점을 분석한 결과, 사용자 입력과 HTTP 요청을 신뢰하는 것이 주요 보안 문제로 이어질 수 있음을 확인하였다.

이번 실습에서는 SQL Injection, XSS, Brute Force, 접근제어 취약점, 파일 접근 문제 및 비즈니스 로직 조작 등을 직접 확인하면서 **취약점 발견** → **원인 분석** → **공격 영향 확인** → **대응 방안 도출**의 전체적인 웹 보안 분석 과정을 경험하였다.

이를 통해 실제 모의해킹 과정에서 단순히 취약점의 존재 여부를 확인하는 것이 아니라, 해당 취약점이 시스템과 사용자에게 미치는 영향을 분석하고 적절한 보안 대책까지 제시하는 것이 중요하다는 것을 학습하였다.